

UNIVERSIDAD DE EL SALVADOR

Facultad Multidisciplinaria De Occidente Departamento
de Ingeniería y Arquitectura Ingeniería en Desarrollo de
Software

INGENIERO: ERICK TRIGUEROS



INTEGRANTES:

GRUPO 1: CRM Para Gestion de clientes

- **Alyson Pamela Alvarado Jovel (aj24002)**
- **Josué Mauricio García Ruiz (gr24071)**
- **Luis Daniel Conteras Rivera (cr11010)**

GRUPO 8: Chatbot +IA

- **José Manuel Mendoza Vasquez (mv24005)**
- **José Ángel Pino López (pl21026)**
- **Christopher Bryan Rodríguez Medina (rm21062)**

PROYECTO 2

Agradecimientos

Queremos dedicar este espacio para expresar nuestro más profundo agradecimiento a la **Universidad de El Salvador**, institución que ha sido un pilar fundamental en nuestra formación profesional. A lo largo de nuestro recorrido académico, esta universidad nos ha brindado no solo el conocimiento teórico necesario para enfrentar los retos del mundo profesional, sino también las herramientas prácticas que nos han permitido llevar a cabo este proyecto con éxito.

Agradecemos el compromiso y la visión de la Universidad de El Salvador en fomentar el desarrollo integral de sus estudiantes, impulsándonos constantemente a innovar, a ser agentes de cambio y a aplicar nuestros conocimientos para contribuir al crecimiento de la sociedad salvadoreña. Su esfuerzo por formar profesionales íntegros y competentes ha sido una motivación constante durante este proceso.

Asimismo, queremos extender un agradecimiento especial al **Ingeniero Erick Trigueros**, quien desempeñó un papel clave en la realización de este proyecto. Su guía, paciencia y asesoramiento técnico fueron esenciales para superar los desafíos encontrados a lo largo del desarrollo. Su vasta experiencia, combinada con su capacidad de motivarnos y de inculcar en nosotros un enfoque crítico y creativo, marcó una diferencia significativa en nuestro trabajo.

Su dedicación y apoyo incondicional no solo nos ayudaron a mejorar nuestras habilidades técnicas, sino también a fortalecer nuestra confianza y compromiso con la excelencia. Gracias a su orientación, hemos podido consolidar una solución tecnológica que esperamos tenga un impacto positivo en el ámbito empresarial y social.

Este proyecto es el resultado de un esfuerzo colectivo, y no hubiera sido posible sin el respaldo de quienes confiaron en nuestras capacidades y nos acompañaron en este camino. Por ello, reiteramos nuestra gratitud hacia la Universidad de El Salvador, al Ingeniero Erick Trigueros, y a todas las personas que, de una u otra forma, contribuyeron a que este proyecto se materializara.

Introducción

El propósito de este documento es describir el diseño del estudio y las especificaciones técnicas necesarias para el desarrollo de dos proyectos clave:

1. **Sistema de Gestión de Relaciones con Clientes (CRM):** Este proyecto tiene como objetivo crear una aplicación web y móvil que facilite la gestión eficiente de relaciones con los clientes, combinando enfoques cualitativos y cuantitativos en su diseño para garantizar una solución integral y basada en datos.
2. **Chatbot desarrollado desde cero:** Este proyecto se enfoca en el desarrollo de un chatbot utilizando Python y la tecnología Gemini. Está diseñado para mejorar la experiencia del usuario en pequeñas y medianas empresas (pymes) al reducir los tiempos de respuesta a preguntas frecuentes sobre productos o servicios, optimizando la atención al cliente mediante respuestas ágiles y precisas.

El documento está dirigido a los equipos de desarrollo, investigación y a los stakeholders clave involucrados en estos proyectos, proporcionando una guía clara y detallada sobre los aspectos técnicos y estratégicos que sustentan su implementación.

Objetivo General

Desarrollar e implementar soluciones tecnológicas innovadoras, como un sistema de gestión de relaciones con clientes (CRM) y un chatbot basado en inteligencia artificial, que optimicen la gestión de relaciones comerciales y mejoren la experiencia del cliente en pequeñas y medianas empresas (pymes).

Objetivos Específicos

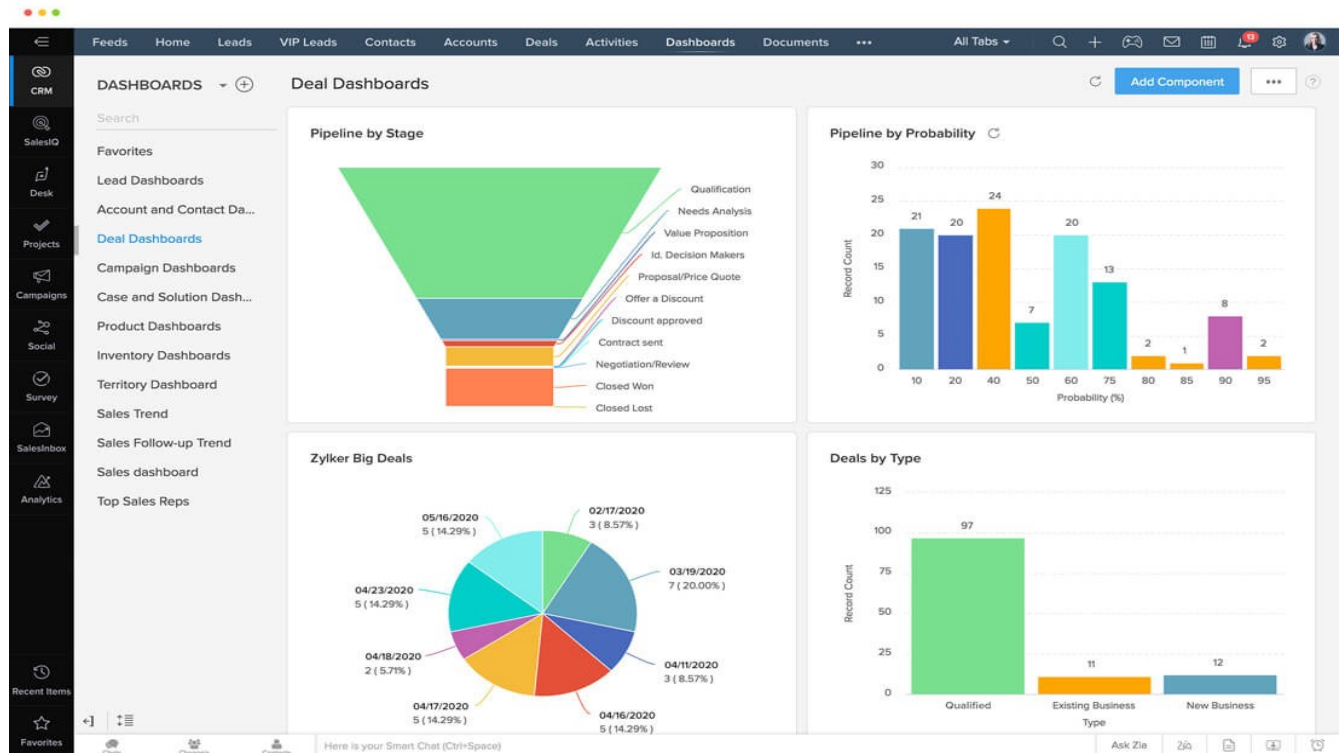
Para el CRM:

1. Diseñar una aplicación web/móvil que integre funcionalidades clave como la gestión de contactos, automatización de marketing y seguimiento de interacciones.
2. Implementar un sistema escalable y seguro mediante el uso de tecnologías modernas como microservicios, encriptación avanzada y almacenamiento en la nube.
3. Incorporar métricas e informes interactivos para la toma de decisiones estratégicas basadas en datos.
4. Asegurar una experiencia de usuario intuitiva mediante una interfaz de usuario optimizada y herramientas de onboarding.

Para el Chatbot:

1. Diseñar e implementar un chatbot utilizando Python y la tecnología Gemini para automatizar respuestas a preguntas frecuentes de clientes.
2. Optimizar la atención al cliente en pymes mediante tiempos de respuesta más ágiles y precisos.
3. Integrar el chatbot con plataformas existentes de las empresas, como sitios web, aplicaciones móviles o sistemas CRM.
4. Garantizar una experiencia conversacional natural mediante técnicas de procesamiento de lenguaje natural (NLP).

Sistema de Gestión de Relaciones con Clientes (CRM)



2. Diseño del Estudio

2.1 Enfoque Metodológico

- **Enfoque Mixto:** Combinación de métodos cualitativos y cuantitativos para abordar tanto la percepción subjetiva como los datos objetivos de los usuarios.
 - **Cualitativo:**
 - **Métodos:**
 - Grupos focales para discutir experiencias con CRM actuales.
 - Entrevistas semiestructuradas con preguntas abiertas orientadas al comportamiento del usuario.
 - **Herramientas:** Grabadoras de voz, software de transcripción y análisis como NVivo o Atlas.ti.

- **Cuantitativo:**
 - **Análisis:**
 - Diseño de encuestas estructuradas con escalas de Likert.
 - Uso de herramientas como Google Forms, SurveyMonkey o Qualtrics para recopilación digital.
 - **Procesamiento:** Análisis estadístico utilizando software como SPSS, R o Python (bibliotecas: pandas, numpy, matplotlib).

2.2 Herramientas y Técnicas de Recolección de Datos

- **Encuestas:**
 - Diseñadas para recolectar datos demográficos, uso del CRM y percepción del marketing personalizado.
 - Administración digital mediante plataformas seguras con validación de respuestas.
 - Exportación a formatos compatibles (CSV/Excel) para análisis posterior.
- **Entrevistas:**
 - Selección intencional de participantes representativos, utilizando criterios predefinidos (rol, experiencia con CRM).
 - Métodos presenciales o virtuales con herramientas como Zoom o Google Meet.
 - Almacenamiento en bases de datos estructuradas para análisis cualitativo.
- **Análisis de Datos Existentes:**
 - Fuentes: Informes de la industria, repositorios de datos abiertos y APIs públicas.
 - Uso: Integración con herramientas de visualización como Tableau o Power BI.

2.3 Población y Muestra

- **Población:** Empresas que utilizan CRM y sus clientes finales.
- **Muestra:** Selección intencional de:
 - Usuarios actuales de CRM (usuarios empresariales).
 - Profesionales de marketing con experiencia en CRM.

- Clientes finales que interactúan regularmente con estas herramientas.

3. Desarrollo de la Aplicación

3.1 Requerimientos Funcionales

- **Gestión de Contactos:**
 - CRUD (Crear, Leer, Actualizar, Eliminar) con validación en tiempo real.
 - Clasificación de contactos mediante etiquetas personalizadas.
 - Soporte para importación/exportación de datos en formatos CSV y Excel.
- **Seguimiento de Interacciones:**
 - Registro automático de interacciones basadas en actividades (correos, llamadas).
 - Notificaciones programadas utilizando servicios de colas (ej., RabbitMQ o Kafka).
 - Generación de informes PDF dinámicos.
- **Automatización de Marketing:**
 - Workflows programables para campañas.
 - Integración con APIs de terceros para correos masivos (SendGrid, Mailchimp).
 - Tablero de métricas con gráficos interactivos.

3.2 Requerimientos No Funcionales

- **Seguridad:**
 - Encriptación de datos sensibles mediante AES-256.
 - Autenticación con OAuth 2.0 y soporte para SSO.
 - Políticas de roles con permisos jerárquicos.
- **Usabilidad:**
 - Diseño basado en principios de UX/UI utilizando frameworks como Material Design.
 - Onboarding dinámico con tutoriales interactivos.
 - Documentación accesible para desarrolladores y usuarios.
- **Escalabilidad:**

- Arquitectura basada en microservicios para garantizar modularidad.
- Optimización de consultas SQL mediante índices y normalización de tablas.

4. Arquitectura Técnica

4.1 Tecnologías Utilizadas

- **Lenguajes de Programación:**
 - Backend: Python (Django/Flask), Node.js
 - Frontend: JavaScript/TypeScript con React o Angular.
- **Bases de Datos:**
 - Relacional: MySQL/PostgreSQL con soporte para particionamiento.
 - NoSQL: MongoDB para datos no estructurados.
- **Servicios en la Nube:** AWS Lambda para procesos event-driven, Firebase para autenticación y almacenamiento.

4.2 Infraestructura

- **Frontend:** Aplicación SPA con integración de gráficos dinámicos usando D3.js o Chart.js.
- **Backend:** APIs RESTful con soporte para GraphQL para consultas avanzadas.
- **Despliegue:** Uso de contenedores Docker con orquestación en Kubernetes.

5. Plan de Implementación

- **Fase 1:**
 - Definición de requisitos con stakeholders y validación de prototipos iniciales.
 - Diseño del modelo ER de la base de datos.
- **Fase 2:**
 - Prototipado en Figma o Adobe XD para la interfaz.
 - Desarrollo incremental del frontend con metodología ágil (Scrum).
- **Fase 3:**
 - Configuración del entorno backend e integración con APIs de terceros.

- Pruebas unitarias con Jest/Mocha y cobertura del código.
- **Fase 4:**
 - Pruebas de integración de bases de datos y sistema completo.
 - Generación de reportes de desempeño utilizando JMeter.
- **Fase 5:**
 - Implementación de seguridad avanzada y optimización de rendimiento.
- **Fase 6:**
 - Despliegue continuo en AWS con CI/CD usando Jenkins o GitHub Actions.

PREGUNTAS:

1. **¿Cuándo una campaña personalizada les resulta útil y cuándo puede resultar invasiva? (Respondo: Daniel Contreras)**

Una campaña personalizada puede ser útil cuando:

Releva las necesidades del cliente: Si está basada en datos relevantes, como preferencias de compra, historial de navegación o comportamiento, ofrece contenido o productos que realmente interesan al usuario.

Aporta valor: Cuando la personalización se utiliza para mejorar la experiencia del cliente, como ofrecer descuentos, recomendaciones útiles o información relevante en el momento oportuno.

Respetar el momento y el canal: Si se envía en el momento adecuado, a través del canal preferido por el cliente, como email, mensajes en redes sociales o notificaciones.

Una campaña personalizada puede resultar invasiva cuando:

No respeta la privacidad: Si se basa en el uso indebido de datos personales o si el cliente no ha dado su consentimiento explícito para recibir mensajes personalizados.

Es demasiado frecuente: Cuando las comunicaciones son excesivas, el cliente puede sentirse abrumado o molesto.

Falla en el contexto: Si la personalización se siente forzada o irrelevante, como ofrecer productos que el cliente ya compró o no necesita.

2. ¿Qué preocupaciones tienen en cuanto a la seguridad de los datos en un CRM? (Respondio: Alyson Alvarado)

- **Acceso no autorizado**
- **Preocupación:** Que personas no autorizadas, como hackers o ex empleados, puedan acceder a los datos del sistema.
- **Por qué:** Un CRM almacena información clave, como contactos, ventas, estrategias y datos personales, lo cual es valioso y sensible.
- **Solución esperada:** Autenticación de múltiples factores (MFA), contraseñas seguras, y permisos configurados por roles.
- **Filtraciones de datos**
- **Preocupación:** Que los datos almacenados sean robados y publicados o vendidos.
- **Por qué:** Las filtraciones dañan la reputación de la empresa y pueden generar multas si se violan leyes de protección de datos como el RGPD o la Ley de Protección de Datos de El Salvador.
- **Solución esperada:** Cifrado de datos, tanto en tránsito como en reposo, y auditorías regulares de seguridad.
- **Ransomware y ataques cibernéticos**
- **Preocupación:** Que el CRM sea atacado y los datos sean secuestrados por malware (ransomware), exigiendo un pago para recuperarlos.
- **Por qué:** Este tipo de ataque puede paralizar las operaciones y generar grandes pérdidas económicas.

- **Solución esperada:** Copias de seguridad regulares, sistemas de detección de malware y servidores protegidos con firewalls.
- **Vulnerabilidades en integraciones**
 - **Preocupación:** Que las conexiones del CRM con otros sistemas (por ejemplo, tiendas en línea o aplicaciones de correo) tengan fallos que expongan los datos.
 - **Por qué:** Las APIs mal configuradas o sin seguridad pueden ser un punto débil para los atacantes.
 - **Solución esperada:** APIs con autenticación, límites de acceso y cifrado.
- **Pérdida de datos por errores**
 - **Preocupación:** Que los datos se borren o modifiquen accidentalmente, ya sea por fallos técnicos o errores humanos.
 - **Por qué:** La pérdida de datos puede afectar la continuidad del negocio y reducir la confianza de los clientes.
 - **Solución esperada:** Función de "deshacer", registros de cambios y copias de seguridad automáticas.
- **Dependencia del proveedor**
 - **Preocupación:** Si el proveedor del CRM sufre un ataque o quiebra, los datos podrían perderse o quedar comprometidos.
 - **Por qué:** Muchas empresas confían completamente en el proveedor para proteger sus datos.
 - **Solución esperada:** Contratos claros sobre la propiedad y recuperación de datos, además de opciones de exportación segura.
- **Cumplimiento de normativas legales**
 - **Preocupación:** Que el CRM no cumpla con las leyes locales e internacionales de protección de datos.
 - **Por qué:** Esto podría causar multas, pérdida de confianza de los clientes y problemas legales.
 - **Solución esperada:** Funciones que permitan cumplir con regulaciones como RGPD, HIPAA o normativas locales en El Salvador.

3. ¿Cómo reaccionarían si un CRM ofreciera funciones avanzadas de seguridad, como autenticación de múltiples factores? (Respodio: Jose Ruiz)

La reacción a un CRM que ofrece funciones avanzadas de seguridad, como la autenticación de múltiples factores

- Confianza y tranquilidad: La seguridad mejorada inspira confianza en que sus datos están protegidos contra accesos no autorizados.
- Posible resistencia inicial: Algunos usuarios podrían considerar el MFA como un paso adicional que complica el inicio superarse cuando entienden los beneficios.

CHATBOT + IA



Lenguajes de programación: Python. Por su alta compatibilidad para el desarrollo de herramientas de inteligencia artificial, APIs y frameworks.

Paradigmas: Se explora la implementación de 2 paradigmas de programación para el desarrollo de este chatbot:

- Imperativo: Para la conexión con bases de datos y otros servicios necesarios.
- Funcional: Para la lectura de datos y el filtrado de los mismos en los archivos de FAQs, prioritariamente el paradigma imperativo servirá para el resto del desarrollo.

Modelos de inteligencia artificial propuestos:

- Gemini (Google). La opción más viable para la implementación de servicios de Google Cloud.
- GPT-4 (OpenAI). De los mejores modelos en el mercado, desarrollo robusto y alta personalización.
- Claude Haiku (Anthropic). Enfoque más conversacional y ético.

Inicialmente se consideró Gemini 1.5 de Google, pero debido a nuevos costos asociados, se puede tomar en cuenta el uso de GPT-4o de OpenAI,

Frameworks:

En base a que nuestra aplicación es de carácter I/O y depende en gran medida de la espera al leer respuestas en un banco de preguntas en base de datos o en el tiempo de procesamiento de la IA, se tomará en cuenta las siguientes herramientas:

- Backend: FastAPI.
- Servidores de producción: Uvicorn para las peticiones de los usuarios y Gunicorn como puerta de enlace HTTP. NGINX como proxy reverso para mejorar la seguridad de la aplicación. Y Google Compute Engine para alojamiento.

Está potente combinación entre FastAPI + Uvicorn ofrece un consumo inferior de recursos de procesamiento.

Para los requerimientos de alojamiento se consideran los siguientes:

Base de datos:

- PostgreSQL De carácter relacional, servirá para el almacenamiento de las FAQs en la base de datos.
- Redis: Para el almacenamiento del caché de la aplicación.

Utilizaremos Redis para evaluar si hay FAQs almacenadas en el caché y de esta manera ahorrar tiempo de respuesta y evitar el consumo de la API de las IA.

Integración de la API Inteligencia artificial:

- Hugging Face Hub, para OpenAI (GPT-4).
- Google SDK, para Gemini.

Para ello se necesita la importación de la librería:
`google.generativeai`

Escalabilidad:

- Docker: Para despliegue rápido.
- Kubernetes: Para escalabilidad horizontal. (Opcional pero bastante considerable a implementar a futuro).

Frontend:

- El chatbot se presenta como un widget desarrollado con HTML, CSS y JavaScript. Su función será la integración embebida del chatbot en la página web del usuario.
- React.js, actuará como el framework de JavaScript para darle una interfaz más moderna y atractiva.

¿Cómo funciona el Chatbot?

1. Recepción de consulta:

- La API recibe el texto del cliente a través del widget incrustado en la web, escribiendo en una caja de texto.

2. Validación de entrada:

- Procesar la consulta para evitar entradas dañinas o vacías (sanitización básica). Gracias a Pandas para el tratamiento y sanitización de datos.

3. Búsqueda en la base de conocimientos (FAQ):

- Consultar FAQs en la base de datos para ofrecer una respuesta rápida y directa, estas estarán almacenadas en la base de datos usando PostgreSQL o en caché almacenado usando Redis.

4. Consulta al modelo IA (si es necesario):

- Si no se encuentra respuesta en las FAQs, redirigir la consulta al modelo de IA seleccionado.
- Mantener un registro de consultas no respondidas para mejorar las FAQs en el futuro (Necesario considerar implementación).

5. Devolución de respuesta:

- Mostrar la respuesta al cliente en el widget del sitio web.

7. Construcción de la API

Rutas principales:

- **/query (POST)**: Recibe una consulta del cliente.
- **/update-faq (POST)**: Endpoint para actualizar la base de datos de FAQs.
- **/health-check (GET)**: Para monitoreo del estado del servidor.

8. Seguridad y rendimiento

1. Seguridad:

- Implementar HTTPS con certificados SSL.
- Autenticación por token en las APIs (JWT o OAuth 2.0).

2. Rendimiento:

- Caché: Usar Redis para almacenar respuestas frecuentes.

9. Embedding en el sitio web

- Usar JavaScript para integrar el widget personalizado en la web y que esta consuma la API del chatbot.

10. Pruebas

Pruebas:

- Usar Pytest para probar funcionalidades backend.
- Simulación de conversaciones con POSTMAN.

PREGUNTAS

1. ¿Qué criterios utilizaron para elegir entre Gemini y GPT-4 como modelo de IA principal? (Respondio: Angel Pino)

Se seleccionó Gemini inicialmente por su integración con Google Cloud, pero debido a los costos, GPT-4 también es una opción viable por su personalización robusta y alto rendimiento.

2. ¿Qué desafíos potenciales esperan con la integración de Redis para la caché y cómo los resolverán? (Respondio: Jose Mendoza)

Los desafíos incluyen la coherencia de datos y el mantenimiento del almacenamiento en caché actualizado. Esto se aborda implementando estrategias de invalidación de caché y sincronización regular con PostgreSQL.

3. ¿Cuáles son los criterios para decidir entre PostgreSQL y Redis para el almacenamiento de datos? (Respondio: Christopher Rodriguez)

PostgreSQL se usa para almacenamiento permanente y estructurado, mientras que Redis se usa para respuestas frecuentes en caché, mejorando el tiempo de respuesta.